

Contact Manager — ATTSW Project Report

Student: Mohamed Jama Farah

Professor: Lorenzo Bettini

University: University of Florence

Course: ATTSW — A.Y. 2025/2026

Exam date: July 20, 2026, 09:30, Aula 005 C.Didat.Morgagni

GitHub: <https://github.com/mohamedjama-farah/Contact-Manager>

1. Introduction

This report describes the Contact Manager application I built for the ATTSW course. The application lets the user add, view, update, and delete contacts, and it stores them in MongoDB.

I wrote the project with Test-Driven Development. I did not write a class or a method before a failing test asked for it, and the git history shows the Red and Green commits for each feature. The project uses unit tests, integration tests, and an end-to-end test. Coverage is kept at 100% for lines and branches with JaCoCo, and the project also uses PIT for mutation testing, SonarCloud for static analysis, Coveralls for the coverage badge, and GitHub Actions for continuous integration.

2. Architecture (MVP)

I used the Model-View-Presenter pattern. The controller only depends on two interfaces, `ContactRepository` and `ContactView`, so I can test it with Mockito mocks and no database or GUI. The real classes, `MongoContactRepository` and `ContactFrame`, are put together only in `ContactManagerApp` when the application starts.

Layer	Class / Interface	Role
Model	<code>Contact</code>	Holds id, name, phone, email
Repository	<code>ContactRepository</code>	Interface: save, findAll, findById, delete, update
Repository	<code>MongoContactRepository</code>	MongoDB implementation
Controller	<code>ContactController</code>	Business logic between repository and view
View	<code>ContactView</code>	Interface: showAllContacts, showError, contactAdded, contactDeleted, contactUpdated
View	<code>ContactFrame</code>	Swing implementation
App	<code>ContactManagerApp</code>	<code>main()</code> : reads command-line options and wires the parts together

I excluded `ContactManagerApp` from coverage because its `main()` only reads options and wires the parts, so there is nothing meaningful to unit test in it.

3. Test-Driven Development

Every production class started from a failing test. For each feature I first wrote a test that failed, then wrote the smallest amount of code to make it pass, then cleaned up if there was anything to clean. You can see this in the history, for example the commit `RED: add failing test for updating a contact` followed by `GREEN: implement updateContact`, and `RED: add failing test for empty-name validation on add` followed by `GREEN: reject empty name on add`.

4. Operations (CRUD)

The application does all four CRUD operations on a contact.

- **Create:** `addContact(name, phone, email)`. Before it saves, the controller checks the input. It refuses an empty or blank name, and it refuses a contact whose phone number already exists in the stored contacts. In both cases the view shows an error and nothing is saved.
- **Read:** `allContacts()` loads all contacts and shows them in the list.
- **Update:** `updateContact(id, name, phone, email)`. The controller finds the contact by id first. If it is not there, it shows an error. If it is there, it updates the stored record. The user reaches this through the Update button.
- **Delete:** `deleteContact(id)` removes the selected contact.

5. Tests

The project has three kinds of tests.

5.1 Unit tests (Surefire)

`ContactControllerTest` has 9 tests. It checks the controller logic with Mockito mocks for the repository and the view: list all, add, add with empty name, add with null name, add with a duplicate phone, add with a different phone, delete, update, and update when the contact does not exist.

`ContactTest` has 1 test for the model.

`ContactFrameTest` has 8 tests for the Swing GUI with AssertJ-Swing. I create the frame on the Event Dispatch Thread with `GuiActionRunner.execute(...)`, wrap it in a `FrameFixture`, and mock the controller. The tests cover showing contacts, the error dialog, the Add button, the Update button, the Delete button, the Delete and Update buttons being disabled when nothing is selected, the two buttons becoming enabled once a contact is selected, and the list refreshing after an update.

5.2 Integration tests (Failsafe, files ending in IT)

`MongoContactRepositoryIT` has 7 tests. It runs the MongoDB repository against a real MongoDB 6.0 container started by Testcontainers: `save` and `findAll`, the generated id, `findById` when the contact exists and when it does not, `update`, `delete`, and `close`. I empty the collection before each test so the tests do not affect each other.

`ContactControllerIT` has 3 tests. It wires the real controller to the real `MongoContactRepository`, with no mocking, and checks that adding, deleting, and updating a contact really change the database.

5.3 End-to-end test (Failsafe, separate execution)

`ContactManagerE2E` has 3 tests and runs the whole application through its main file. It starts `ContactManagerApp` with `ApplicationLauncher.application(...).withArgs("--mongo-host=...", "--mongo-port=...").start()`, so it drives the running application the way a user would and does not call the application classes directly. The three tests add a contact, add then delete, and add then update. This test lives in its own `src/e2e` folder and runs in a separate Failsafe execution, apart from the integration tests.

Level	Files	Tools	Needs
Unit	<code>ContactControllerTest</code> , <code>ContactTest</code> , <code>ContactFrameTest</code>	JUnit 4, Mockito, AssertJ-Swing	Mocks only, virtual screen for the GUI
Integration	<code>MongoContactRepositoryIT</code> , <code>ContactControllerIT</code>	JUnit 4, Testcontainers	MongoDB in Docker
End-to-end	<code>ContactManagerE2E</code>	AssertJ-Swing ApplicationLauncher, Testcontainers	The whole app through main, plus the database

6. Build with Maven

Maven compiles the code, resolves dependencies, and runs the tests.

Surefire runs the unit tests (files ending in `Test`) and skips the `IT` files. The build-helper plugin adds `src/it/java` and `src/e2e/java` as extra test-source folders. Failsafe runs the integration and end-to-end tests in two separate executions: one for the `IT` files and one for the `E2E` file. JaCoCo prepares its agent, writes the report, and fails the build if line or branch coverage falls below 100%, with `ContactManagerApp` excluded. PIT runs mutation testing on the controller. SonarCloud and Coveralls send their reports to their dashboards.

6.1 Excluding a SonarQube rule in the POM

`testShowErrorDisplaysDialog()` checks the result with AssertJ-Swing's `requireMessage(...)`, which is a real assertion, but Sonar's rule "Tests should include assertions" (java:S2699) does not recognise it. Instead of adding a fake assertion to the test, I excluded that rule for that one test file in `pom.xml` with `sonar.issue.ignore.multicriteria`.

6.2 Enabling the buttons instead of guarding inside the listener

The Delete and Update buttons start disabled. A `ListSelectionListener` on the list turns them on when a contact is selected and off when nothing is selected. Because a disabled button cannot be clicked, the button listeners do not need an `if (index != -1)` check, so there is no half-covered branch left to worry about.

7. Continuous Integration (GitHub Actions)

Every push runs `.github/workflows/build.yml` on `ubuntu-latest`. It checks out the repository with full history, sets up Java 17, and runs `xvfb-run mvn verify coveralls:report -DrepoToken=$COVERALLS_REPO_TOKEN`. `xvfb-run` gives the GUI and end-to-end tests a virtual display, and `verify` runs the whole test pipeline and the coverage check. After that it sets up Java 21 and runs `mvn sonar:sonar` for the SonarCloud analysis. The runners already have Docker, so Testcontainers starts MongoDB with no extra setup.

8. Test summary

Test class	Type	Count	Runner
ContactControllerTest	Unit	9	Surefire
ContactTest	Unit	1	Surefire
ContactFrameTest	Unit (GUI)	8	Surefire
MongoContactRepositoryIT	Integration	7	Failsafe
ContactControllerIT	Integration	3	Failsafe
ContactManagerE2E	End-to-end	3	Failsafe
Total		31	

JaCoCo keeps line and branch coverage at 100%.

9. Source structure

```
contact-manager/  
|-- pom.xml
```

```
| -- .github/workflows/build.yml
| -- README.md
| -- src/
|   |-- main/java/com/example/contactmanager/
|   |   |-- app/ContactManagerApp.java
|   |   |-- controller/ContactController.java
|   |   |-- model/Contact.java
|   |   |-- repository/ContactRepository.java
|   |   |-- repository/MongoContactRepository.java
|   |   |-- view/ContactView.java
|   |   |-- view/swing/ContactFrame.java
|   |-- test/java/com/example/contactmanager/
|   |   |-- controller/ContactControllerTest.java
|   |   |-- model/ContactTest.java
|   |   |-- view/swing/ContactFrameTest.java
|   |-- it/java/com/example/contactmanager/
|   |   |-- repository/MongoContactRepositoryIT.java
|   |   |-- controller/ContactControllerIT.java
|   |-- e2e/java/com/example/contactmanager/
|   |   |-- e2e/ContactManagerE2E.java
```

9.1 Repository

<https://github.com/mohamedjama-farah/Contact-Manager->

The git history follows the TDD steps: each feature has its Red and Green commits.

10. Conclusion

The project puts together the techniques from the course: TDD for all the production code, the MVP pattern, three levels of tests, 100% coverage that the build enforces, mutation testing, static analysis, and continuous integration.

Goal	Done
TDD for all production classes	Yes
MVP pattern	Yes
Full CRUD, including Update	Yes
Unit, integration, and end-to-end tests	Yes (31 tests)
End-to-end test through main	Yes
Integration tests for the repository and the controller	Yes
100% line and branch coverage (JaCoCo)	Yes
Mutation testing (PIT)	Yes
Static analysis (SonarCloud)	Yes
Continuous integration (GitHub Actions)	Yes